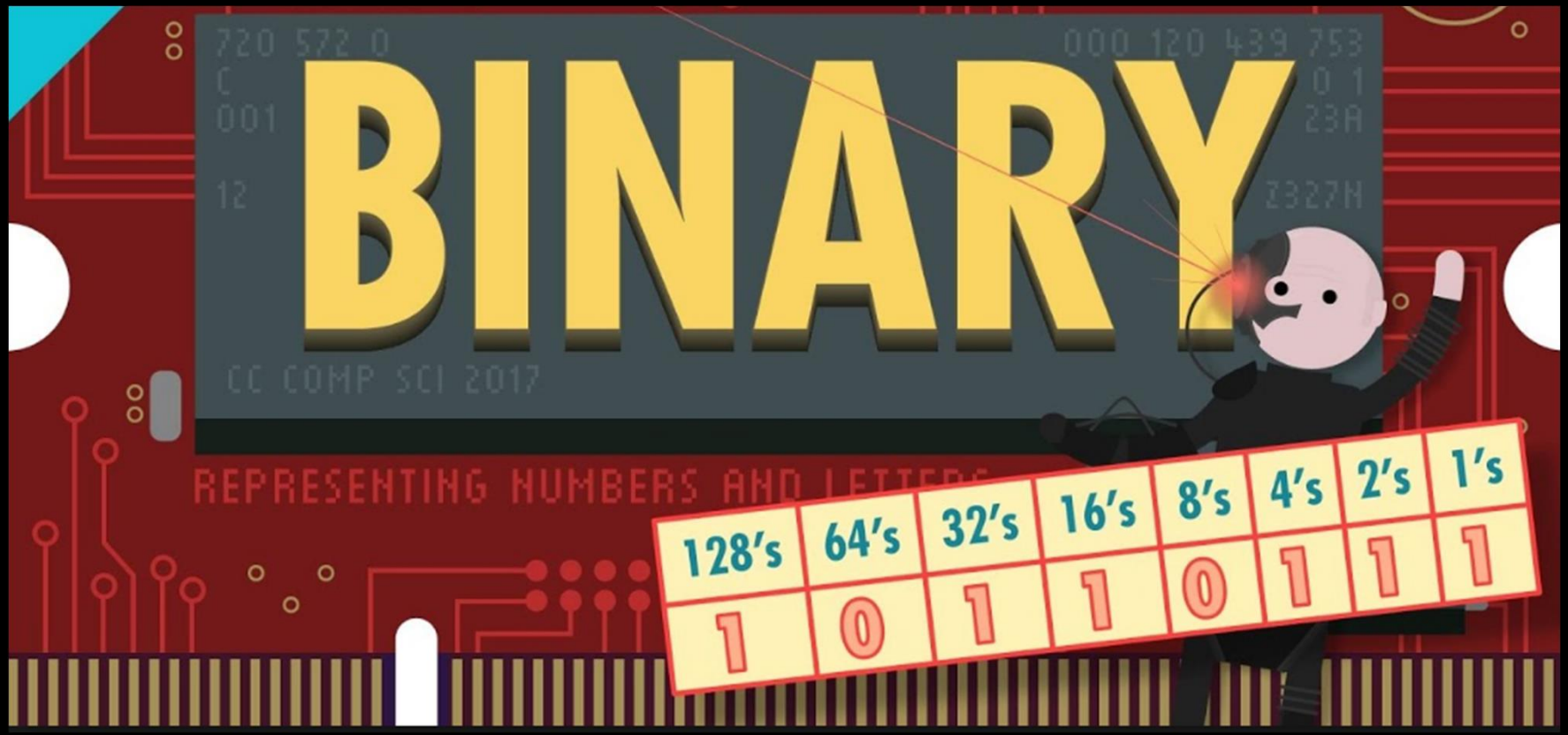
Representação da Informação



Representação da Informação

- A organização de computadores e sistemas digitais é dependente da forma como são representadas informações como números, caracteres e sinais de controle

- Padrões, convenções e normas estabeleceram ao longo dos anos determinaram vários aspectos da organização dos computadores

```
4  {
5  signed char x;
6  x = -3 + -128;
7  printf ("numero %d ", x);
8  }
```

Console

```
:\MinGW\bin\gcc "C:\intranet\ufs\disciplinas\2010_2\103431_circutos_digitais_1\docs\tipos_dados.c" -o "C:\intranet\ufs\disciplinas\20
Process started >>>
:\intranet\ufs\disciplinas\2010_2\103431_circutos_digitais_1\docs\tipos_dados.c: In function `main':
:\intranet\ufs\disciplinas\2010_2\103431_circutos_digitais_1\docs\tipos_dados.c:6: warning: overflow in implicit constant conversion
<<< Process finished.
================= READY =================
```

Sistemas numéricos

Formas de representar números:

- Bases: Decimal, Octal, Hexadecimal, Binária

- Bases:
  - Decimal representada por 10 dígitos (0,1,2,3,4,5,6,7,8,9)
  - Binária representada por 2 dígitos (0,1)
  - Hexadecimal (0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F)

Sistemas numéricos

Conversão entre bases numéricas:

- 10010110 – binário
- 150 – decimal
- 96 - hexadecimal

Sistemas numéricos

Conversão entre bases numéricas:

• Decimal para Binário

Dividir o valor em decimal por dois até obter quociente zero

O valor binário é dados pelos valores dos restos lidos na ordem inversa

Sistemas numéricos

Conversão entre bases numéricas:

- Decimal para Binário
- Ex.:

```
26 | 2
0   13 | 2
     1   6 | 2
          0   3 | 2
               1   1 | 2
                    1   0
```

- 11010_2

Sistemas numéricos

Conversão entre bases numéricas:

- Binário para Decimal

$$10110_2 = 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 22_{10}$$

$$16 + 0 + 4 + 2 + 0$$

Sistemas numéricos: Hexadecimal

Hexadecimal	Decimal
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
A	10
B	11
C	12
D	13
E	14
F	15

Sistemas numéricos

Decimal	*Binary*	*Octal*	*Hexadecimal*
0	00000000	000	00
1	00000001	001	01
2	00000010	002	02
3	00000011	003	03
4	00000100	004	04
5	00000101	005	05
6	00000110	006	06
7	00000111	007	07
8	00001000	010	08
9	00001001	011	09
10	00001010	012	0A
11	00001011	013	0B
12	00001100	014	0C
13	00001101	015	0D
14	00001110	016	0E
15	00001111	017	0F
16	00010000	020	10
17	00010001	021	11
18	00010010	022	12
:	:	:	:
etc.	etc.	etc.	etc.

Sistemas numéricos

Conversão entre bases numéricas:

- Binário para Hexadecimal (método de substituição direta)

011100001100_2

0111 0000 1100

7 0 C $\rightarrow$ $70C_{16}$

Sistemas numéricos

Conversão entre bases numéricas:

- Hexadecimal para Decimal

$$E1A_{16} = 14 \times 16^2 + 1 \times 16^1 + 10 \times 16^0 = 3610_{10}$$

$$3584 + 16 + 10$$

Aritmética Binária

- Soma, Subtração
- Representação de números negativos
- Complementos de 1 e 2
- Soma e Subtração com Complemento de Dois

Aritmética Binária: Soma

Regras para adição de números binários

- $0 + 0 = 0$
- $0 + 1 = 1$
- $1 + 1 = 0$ com transporte de 1
- $1 + 1 + 1 = 1$ com transporte de 1

Aritmética Binária: Soma

• Soma-se as colunas e realiza o transporte quando necessário

1 1 0 1 ——— Transporte

1 1 0 1

+1 1 0 1

1 1 0 1 0

Regras:

• 0 + 0 = 0

• 0 + 1 = 1

• 1 + 1 = 0 c/ transporte

• 1 + 1 + 1= 1 c/ transporte

Aritmética Binária: Subtração

Regras para subtração de números binários

- 0 - 0 = 0
- 1 - 1 = 0
- 1 - 0= 1
- 0 - 1= 1 empresta 1

Aritmética Binária: Subtração

$$\begin{array}{r} \cancel{1}\cancel{1}011 \\ -\ \underline{01101} \\ 01110 \end{array}$$

Regras

0 - 0 = 0

1 - 1 = 0

1 - 0= 1

0 - 1= 1 empresta 1

Aritmética Binária: Representação de Números Negativos

Sinal e Magnitude

$+ 45_{10}$ => $\underline{0}$ $\underline{0101101}$

Sinal

Magnitude

$- 45_{10}$ => $\underline{1}$ $\underline{0101101}$

O bit mais significativo indica o sinal e os remanescentes indicam o valor. Circuito aritmético complexo para execução.

Aritmética Binária: Representação de Números Negativos

Complemento de Um

$+ 4_{10}$ => 0 0000100

Sinal

Magnitude

$- 4_{10}$ => 1 1111011

Positivos: O bit mais significativo corresponde ao sinal. Bits remanescentes contém o valor

Negativos: inversão de todos bits incluindo bit de sinal

Aritmética Binária: Representação de Números Negativos

Complemento de Dois

- Positivos: mesma representação do modo Sinal Magnitude e Complemento de Um

4_{10} => 00000100_2

- Negativos: Converter para Complemento de Um e somar o valor 1

- 4_{10} => 11111100_2

Aritmética Binária: Representação de Números Negativos

Complemento de Dois

- 4_{10} => 00000100 (binário sem sinal)

• Complemento de Um => 11111011

• Complemento de Dois +1

11111100

Aritmética Binária: Representação de Números Negativos

Complemento de Dois

- 17_{10} => 00010001 (binário sem sinal)

• Complemento de Um => 11101110

• Complemento de Dois +1

11101111

Aritmética Binária: Representação de Números Negativos

Complemento de Dois – Operação Subtração e Soma

$$
\begin{array}{ll}
 & 00000111_2 \quad (7_{10}) \\
+ & \underline{11111101}_2 \quad (-3_{10}) \\
 & 1000000100 \quad (4_{10})
\end{array}
$$

$\downarrow$

Ignorar transporte final

Aritmética Binária: Representação de Números Negativos

Complemento de Dois – Operação Subtração e Soma

$$
\begin{array}{rll}
 & 11110111_2 & (-9_{10}) \\
+ & \underline{00001000}_2 & (8_{10}) \\
 & 11111111 & (-1_{10})
\end{array}
$$

Aritmética Binária: Representação de Números Negativos

Complemento de Dois – Operação Subtração e Soma

$$
\begin{array}{rll}
 & 11111101_2 & (-3_{10}) \\
+ & \underline{11111100}_2 & (-4_{10}) \\
1 & 11111001 & (-7_{10})
\end{array}
$$

$\downarrow$

Ignorar transporte final

Aritmética Binária: Representação de Números Negativos

Complemento de Dois – Operação Subtração e Soma

$$
\begin{array}{rll}
 & 10001000_2 & (-120_{10}) \\
+ & \underline{11101110}_2 & (-18_{10}) \\
 & 1\ 01110110 & \\
 & \downarrow & \\
 & \text{Overflow (transbordo)} &
\end{array}
$$

Circuitos Digitais I

Aritmética Binária: Representação de Números Negativos

Aplicação em programação

Type	Bytes	Bits	Range	
short int	2	16	-32,768 -> +32,767	(16kb)
unsigned short int	2	16	0 -> +65,535	(32Kb)
unsigned int	4	16	0 -> +4,294,967,295	(4Gb)
int	4	32	-2,147,483,648 -> +2,147,483,647	(2Gb)
long int	4	32	-2,147,483,648 -> +2,147,483,647	(2Gb)
signed char	1	8	-128 -> +127	
unsigned char	1	8	0 -> +255	
float	4	32		
double	8	64		
long double	12	96		

```
4  {
5  signed char x;
6  x = -3 + -128;
7  printf ("numero %d ", x);
8  }
```

Console

```
:\MinGW\bin\gcc "C:\intranet\ufs\disciplinas\2010_2\103431_circutos_digitais_1\docs\tipos_dados.c" -o "C:\intranet\ufs\disciplinas\20
Process started >>>
:\intranet\ufs\disciplinas\2010_2\103431_circutos_digitais_1\docs\tipos_dados.c: In function `main':
:\intranet\ufs\disciplinas\2010_2\103431_circutos_digitais_1\docs\tipos_dados.c:6: warning: overflow in implicit constant conversion
<<< Process finished.
================ READY ================
```

Circuitos Digitais I

Aritmética Binária: Representação em Complemento de Dois

Converta os números para binário com representação em complemento de dois e realize a soma (8 bits proposto)

(-3) + (-128)

Em decimal => - 131

Representação complemento 2 utiliza 1 bit de sinal, portanto sete bits para magnitude

10000000

sinal 2^7 => 128 , portanto -131 gera overflow

Aritmética Binária: Representação em Complemento de Dois

- 128_{10} => 10000000 (binário sem sinal 8 bits)

- Complemento de Um => 01111111
- Complemento de Dois + 1

10000000

Prova - 1

01111111

Invertendo => 10000000

Aritmética Binária: Representação em Complemento de Dois

- 3_{10} => 00000011 (binário sem sinal 8 bits)

• Complemento de Um => 11111100

• Complemento de Dois +1

11111101

Aritmética Binária: Representação em Complemento de Dois

Operação: (-3) + (-128)

$$
\begin{array}{r}
10000000 \\
+\ 11111101 \\
\hline
101111101
\end{array}
$$

sinal

Determine a faixa de valores (indique na base decimal) que pode assumir um número em binário representado em complemento de 2 com dimensão de 8 dígitos

Converter os números a seguir da base decimal sem sinal para a base numérica binária:

32, 23

Converter os números representados na base binária sem sinal para a base hexadecimal:

100010101, 101, 10011001, 1111011110100111

Converter os números a seguir da base decimal para a base binária com complemento de 1:

6, -9, -17, 33

Converter os números a seguir da base decimal para a base binária com complemento de 2:

12, -15, -34, -44

Converta os números para binário com representação em complemento de dois e realize as somas

7 + 8, -3 + -4, -128 + -3, -56 + 32